

Problem Set 7

Due DateNovember 05, 2024 (11:59 PM)
Name **Hyma Jujjuru**
Student ID **110743269**
Collaborators **List Your Collaborators Here**

Contents

Instructions	1
Honor Code (Make Sure to Virtually Sign)	2
1 Dynamical Programming I	3
1.1 Problem 1 [10 pts]	3
1.2 Problem 2 [10 pts]	4
1.3 Problem 3 [10 pts]	5
1.4 Problem 4 [10 pts]	6
1.5 Problem 5 [20 pts]	7
1.6 Problem 6 [60 pts]	8

Instructions

- The solutions **should be typed**, using proper mathematical notation. We cannot accept hand-written solutions. Here's a short intro to $\text{\LaTeX}$.
- You should submit your work through the **class Gradescope page** only (linked from Canvas). Please submit one PDF file, compiled using this $\text{\LaTeX}$ template.
- You may not need a full page for your solutions; pagebreaks are there to help Gradescope automatically find where each problem is. Even if you do not attempt every problem, please submit this document with no fewer pages than the blank template (or Gradescope has issues with it).
- You are welcome and encouraged to collaborate with your classmates, as well as consult outside resources. You must **cite your sources in this document**. **Copying from any source is an Honor Code violation. Furthermore, all submissions must be in your own words and reflect your understanding of the material.** If there is any confusion about this policy, it is your responsibility to clarify before the due date.
- Posting to **any** service including, but not limited to Chegg, Reddit, StackExchange, etc., for help on an assignment is a violation of the Honor Code.

- You **must** virtually sign the Honor Code (see Section). Failure to do so will result in your assignment not being graded.

Honor Code (Make Sure to Virtually Sign)]

Problem HC. • My submission is in my own words and reflects my understanding of the material.

- Any collaborations and external sources have been clearly cited in this document.
- I have not posted to external services including, but not limited to Chegg, Reddit, StackExchange, etc.
- I have neither copied nor provided others solutions they can copy.

Agreed (signature here). I agree to the above, Hyma Jujjuru.

□

1 Dynamical Programming I

1.1 Problem 1 [10 pts]

Problem 1. Consider the Stair Climbing problem, defined as follows.

- **Instance:** Suppose we have n stairs, labeled $s_1, \dots, s_n$. Associated with each stair s_k is a number $a_k \geq 1$. At stair s_k , we may jump forward i stairs, where $i \in \{1, 2, \dots, a_k\}$. You start on s_1 .
- **Solution:** The number of ways to reach s_n from s_1 .

Your job is to first identify the recursive structure (i.e., suppose we are solving the subproblem at stair s_k , what precise sub-problems do we need to consider?) and then write down the recurrence that relates the solution of a problem to those of the sub-problems.

Answer. Let's visually the stairs as below.

S_1	S_2	...	S_i	...	S_j	...	S_k	...	S_n
-------	-------	-----	-------	-----	-------	-----	-------	-----	-------

Recursive Structure:

To get to S_k , you add up the ways to get to the stairs that can reach S_k . So, if S_i and S_j can reach S_k then the number of ways to reach S_k = the number of ways to reach S_i + the number of ways to reach S_j .

We know S_i and S_j can reach S_k if $i + a_i \geq k$ and $j + a_j \geq k$

Recurrence Relation:

Using the above, we can form a recurrence relation.

When $n = 1$, there is only one way to reach S_1 from S_1 .

S = all the stairs S_k such that $k + a_k \geq n$

$$SC(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ \sum_{i \in S} SC(i) & \text{if } n \geq 2 \end{cases} \quad (1)$$

□

1.2 Problem 2 [10 pts]

Problem 2. Suppose we have an m -letter alphabet $\Sigma = \{0, 1, \dots, m-1\}$. Let W_n be the set of strings $\omega \in \Sigma^n$ such that ω does not have 00 as a substring. Let $f_n := |W_n|$. Write down an explicit recurrence for f_n , including the base cases. Clearly justify each recursive term.

Answer. Breaking down the problem statement,

Lets assume we have values $f_1, f_2, f_3, \dots, f_n$ which represent the number of strings without the substring '00' for a given n .

Let S be the string such that $S = s_1 \dots s_n$. Then, the string can be broken down as follows.

$$S = [s_1 \dots s_{n-1}] \ 1 \longleftarrow f_{n-1}$$

$$S = [s_1 \dots s_{n-1}] \ 2 \longleftarrow f_{n-1}$$

...

$$S = [s_1 \dots s_{n-1}] \ m-1 \longleftarrow f_{n-1}$$

$$S = [s_1 \dots s_{n-1}] \ 0 \longleftarrow f_{n-1}$$

To consider the strings without the substring '00', we look at the possible values of s_{n-1} :

$$S = [s_1 \dots s_{n-2}] \ 1 \longleftarrow f_{n-2}$$

$$S = [s_1 \dots s_{n-2}] \ 2 \longleftarrow f_{n-2}$$

...

$$S = [s_1 \dots s_{n-2}] \ m-1 \longleftarrow f_{n-2}$$

Using the observation from above, we can build the recurrence relation for f_n :

When $n = 1$, the character can be letter 0 $\rightarrow$ $m-1$ which is m strings.

When $n = 2$, the 2 characters can be letter 0 $\rightarrow$ $m-1$ each which is m^2 strings. And there is 1 '00' string. So a total of $m^2 - 1$ strings.

$$f_n = \begin{cases} m & \text{if } n = 1 \\ m^2 - 1 & \text{if } n = 2 \\ (m-1)f_{n-1} + (m-1)f_{n-2} & \text{if } n \geq 3 \end{cases} \quad (2)$$

□

1.3 Problem 3 [10 pts]

Problem 3. Suppose we have the alphabet $\Sigma = \{x, y\}$. For $n \geq 0$, let W_n be the set of strings $\omega \in \{x, y\}^n$ where ω contains yyy as a substring. Let $f_n := |W_n|$. Write down an explicit recurrence for f_n , including the base cases. Clearly justify each recursive term.

Answer. Breaking down the problem statement,

When $n = 1$, there is only 1 character so the number of substrings with yyy is 0.

When $n = 2$, there are only 2 characters so the number of substrings with yyy is 0.

When $n = 3$, there are 3 character so the number of substrings with yyy is 1.

Lets assume we have values $f_1, f_2, f_3, \dots, f_n$ which represent the number of strings with the substring yyy for a given n .

Let S be the string such that $S = s_1 \dots s_n$. Then, the string can be broken down as follows.

If the last character, s_n , is x then,

$$S = [s_1 \dots s_{n-1}] x \leftarrow f_{n-1}$$

If the last character, s_n is y then

$$S = [s_1 \dots s_{n-2}] x y \leftarrow f_{n-2}$$

If the last 2 elements are yy then

$$S = [s_1 \dots s_{n-3}] x y y \leftarrow f_{n-3}$$

If $S = [s_1 \dots s_{n-3} y y y]$, then there are 2^{n-3} ways because the last 3 elements are yyy so it does not matter what the first $n-3$ elements are because all the substrings will have yyy .

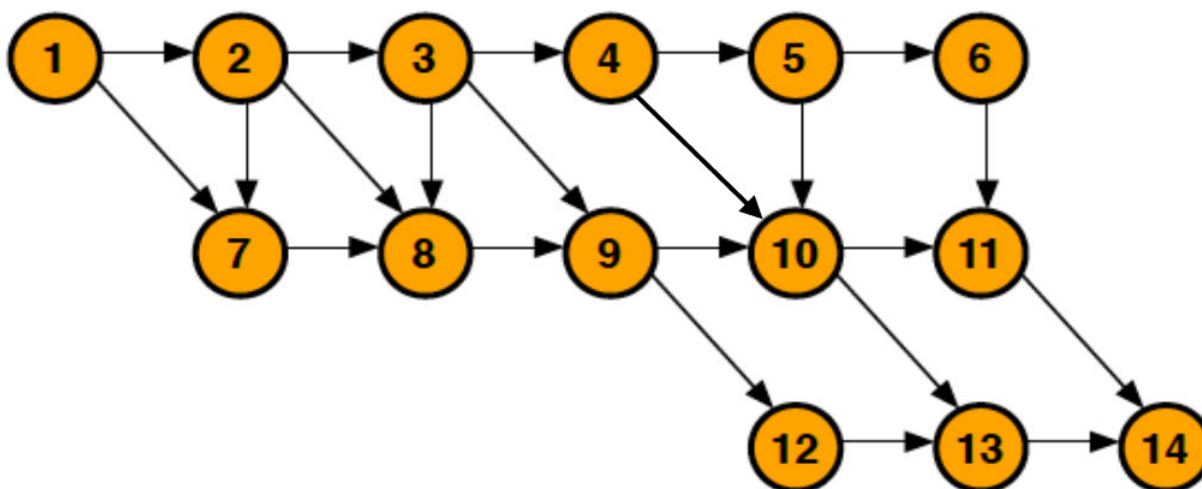
Using the observations from above, we can build the recurrence relation for f_n :

$$f_n = \begin{cases} 0 & \text{if } n = 1 \\ 0 & \text{if } n = 2 \\ 1 & \text{if } n = 3 \\ f_{n-1} + f_{n-2} + f_{n-3} + 2^{n-3} & \text{if } n \geq 4 \end{cases} \quad (3)$$

□

1.4 Problem 4 [10 pts]

Problem 4. Given the following directed acyclic graph. Use dynamic programming to fill in a **one-dimensional** lookup table that counts number of paths from each node j to 14, for $j \geq 1$. Note that a single vertex is considered a path of length 0. **Fill in the lookup table for all vertices 1-14; and in addition, clearly show work for vertices 9-14.**



Answer. **Lookup Table T**

j	1	2	3	4	5	6	7	8	9	10	11	12	13	14
From j To 14	20	17	11	5	3	1	3	3	3	2	1	1	1	1

We can get the number of paths from a node j where j is 1:14 by adding up the number of ways from all the adjacent vertices.

$$T[14] = 1$$

$$T[13] = T[14] = 1$$

$$T[12] = 1$$

$$T[11] = 1$$

$$T[10] = T[11] + T[13] = 1 + 1 = 2$$

$$T[9] = T[10] + T[12] = 2 + 1 = 3$$

We follow this pattern until $T[1]$.

□

1.5 Problem 5 [20 pts]

Problem 5. Consider the following input for the Knapsack problem with a capacity $W = 12$:

item i	1	2	3	4	5	6
value v_i	2	7	18	23	29	35
weight w_i	1	2	5	6	7	9

Fill in a lookup table, similar to the example on page 12 of course notes for week 8 (see Week 8 under “Modules” of the course canvas). In addition, clearly explain how you obtain the maximum values/profits $OPT(6, w)$, $w = 7, 8, 9, 10, 11, 12$.

Answer. **Lookup Table**

	0	1	2	3	4	5	6	7	8	9	10	11	12
$\{\}$	0	0	0	0	0	0	0	0	0	0	0	0	0
$\{1\}$	0	2	2	2	2	2	2	2	2	2	2	2	2
$\{1,2\}$	0	2	7	9	9	9	9	9	9	9	9	9	9
$\{1,2,3\}$	0	2	7	9	9	18	20	25	27	27	27	27	27
$\{1,2,3,4\}$	0	2	7	9	9	18	23	25	30	32	32	41	43
$\{1,2,3,4,5\}$	0	2	7	9	9	18	23	29	31	36	38	41	47
$\{1,2,3,4,5,6\}$	0	2	7	9	9	18	23	29	31	36	38	42	47

Obtaining maximum values/profits $OPT(6, w)$:

Since the first parameter $i = 6$, we look at the last row of the lookup table whose cells represent the highest value the subsets of items can represent within each weight limit.

For $w = 7$, $OPT(6, 7) = 29$

This is derived by adding the value of item 5 (weight 7, value 29)

For $w = 8$, $OPT(6, 8) = 31$

This is derived by adding the values of item 1 (weight 1, value 2) and item 5 (weight 7, value 29).

For $w = 9$, $OPT(6, 9) = 36$

This is derived by adding the values of item 2 (weight 2, value 7) and item 5 (weight 7, value 29).

For $w = 10$, $OPT(6, 10) = 38$

This is derived by adding the values of item 1 (weight 1, value 2), item 2 (weight 2, value 7), and item 5 (weight 7, value 29).

For $w = 11$, $OPT(6, 11) = 41$

This is derived by adding the values of item 2 (weight 2, value 7) and item 6 (weight 9, value 35).

For $w = 12$, $OPT(6, 12) = 47$

This is derived by adding the values of item 3 (weight 5, value 18) and item 5 (weight 7, value 29).

□

1.6 Problem 6 [60 pts]

Problem 6. The sequence F_n of Fibonacci numbers is defined by the recurrence relation

$$F_n = F_{n-1} + F_{n-2} \quad (1)$$

with seed values $F_0 = 1$ and $F_1 = 1$.

1. (10 pts) Consider the recursive top-down implementation of the recurrence (1) for calculating n -th Fibonacci number F_n .
 - (a) Write down an algorithm for the recursive top-down implementation in pseudocode.
 - (b) Write down the recurrence for the running time $T(n)$ of the algorithm and solve for it.

Answer. Pseudocode:

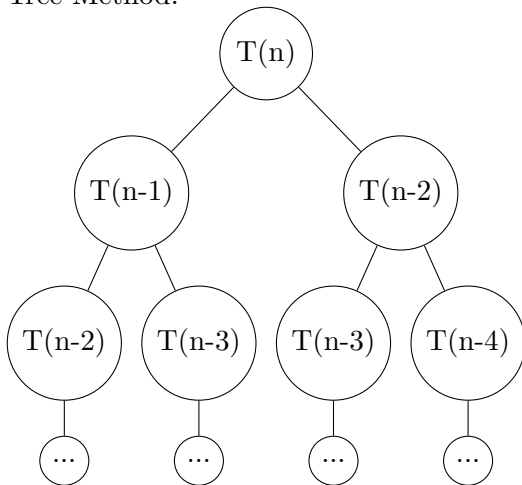
```
procedure FIBONACCI( $n$ )  
  if  $n \leq 1$  then  
    return  $n$   
  return (FIBONACCI( $n - 1$ ) + FIBONACCI( $n - 2$ )))
```

Run Time Recurrence Relation:

$$T(n) = \begin{cases} O(1) & \text{if } n \leq 1 \\ T(n-1) + T(n-2) + O(1) & \text{otherwise} \end{cases} \quad (4)$$

Run Time Complexity:

Tree Method:



Worst Case Levels: n

Worst Case Nodes at Each Level: 2^n

Worst Case Run Time: $\sum_{i=0}^n 2^i = 2^{n+1} - 1$

Run Time Complexity: $T(n) = O(2^n)$

Unrolling Method:

$$\begin{aligned}T(n) &= T(n-1) + T(n-2) + c \\&= T(n-2) + T(n-3) + c + T(n-2) + c \\&= 2T(n-2) + T(n-3) + 2c \\&\leq 2[T(n-2) + T(n-3) + c] \\&= 2T(n-1) \\&= \dots \\&= 2^n T(0) \\&= \Theta(2^n)\end{aligned}$$

Run Time Complexity: $T(n) = O(2^n)$

□

2. (15 pts) Consider the dynamic programming approach “top-down implementation with memoization” that memoizes the intermediate Fibonacci numbers by storing them in an array $F[n]$.
- Write down an algorithm for the top-down implementation with memoization in pseudocode.
 - Describe the behavior of the algorithm in terms of a traversal of a recursion tree. Describe how the array F is filled.
 - Determine the asymptotic running time of the algorithm. Prove your claim is correct by induction on the contents of the array.

Answer. **DP Pseudocode:**

```

procedure FIBDP( $n$ )
  if  $n \leq 1$  then
    return  $n$ 
  if  $F[n] \neq \text{NULL}$  then
    return  $F[n]$ 
   $F[n] = \text{fibDP}(n-1) + \text{fibDP}(n-2)$ 
  return  $F[n]$ 

```

Explanation of algorithm: The recursion tree for the fibDP function is as follows:

The root node is fibDP(n).

From fibDP(n) we branch out to fibDP($n-1$) and fibDP($n-2$).

Each of these branches expands out until the base cases fibDP(0) and fibDP(1).

Then we start populating the array starting from $F[0] = 0$ and $F[1] = 1$.

$n = 2$: we have the values for $F[1]$ and $F[0]$ stored in the array, so we can compute $F[2] = F[1] + F[0]$

$n = 3$: then we compute $F[3] = F[2] + F[1]$

and so on until $F[n] = F[n-1] + F[n-2]$ is assigned.

Run Time Complexity Proof by Induction:

Base Case: For $n = 0$ or $n = 1$, the algorithm runs in $O(1)$ or constant time since we just return a value.

Inductive Step:

Assume the algorithm runs in $O(k)$ for all $n \leq k$.

For $k + 1$:

If $F[k+1]$ is null then the algorithm computes $F[k+1] = \text{fibDP}(k) + \text{fibDP}(k-1)$.

By inductive hypothesis, both fibDP(k) and fibDP($k-1$) take $O(k)$ time.

Therefore, the total run time complexity for fibDP($k+1$) = $O(1) + O(k) + O(k-1) = O(1) + O(k) + O(k) = O(k)$

By induction, the time complexity of the algorithm is $O(n)$.

□

3. (10 pts) Consider the dynamic programming approach “iterative bottom-up implementation” that builds up directly to the final solution by filling the F array in order.
- (a) Write down an algorithm for the iterative bottom-up implementation in pseudocode.
 - (b) Determine the time and space usage of the algorithm. Justify your answers and compare them to the answers in part (2).

Answer. DP Bottom Up Pseudocode:

```
procedure FIBDPBU( $n$ )  
  if  $n \leq 1$  then  
    return  $n$   
   $F[n] \leftarrow$  initialized to 0s  $F[0] = 0$   $F[1] = 1$   
  for  $i = 2$  to  $n$  do  
     $F[i] = F[i-1] + F[i-2]$   
  return  $F[n]$ 
```

Time and Space Usage of Algorithm:

Time Complexity: $O(n)$

Justification:

There is a single loop going from 2 to n , performing constant time operations like addition during each iteration. So the total time taken should be relative to n , which means the total time complexity is $O(n)$.

Space Complexity: $O(n)$

Justification:

There is an array of size $n+1$ to store the Fibonacci numbers. The array takes linear space so the space complexity is $O(n)$.

The time and space complexity of this approach and the approach in part 2 are the same. □

4. (5 pts) If you only want to calculate F_n , you can have an iterative bottom-up implementation with $\Theta(1)$ space usage. Write down an iterative algorithm with $\Theta(1)$ space usage in pseudocode for calculating F_n . Justify your algorithm does have $\Theta(1)$ space usage, and determine its running time.

Answer. **DP Bottom Up $O(1)$ Pseudocode:**

```
procedure FIBFOUR( $n$ )  
  if  $n \leq 1$  then  
    return  $n$   
   $x = 0$   
   $y = 1$   
  for  $i = 2$  to  $n$  do  
     $z = y + x$   
     $x = y$   
     $y = z$   
  return  $y$ 
```

Justification of $\Theta(1)$ Space Complexity:

The algorithm has three variables instead of an array of $n+1$ elements. Regardless of the input n , the variables take up the same amount of space which does not grow or scale. So, the space complexity is $\Theta(1)$.

Run Time Complexity: $O(n)$

Justification: The algorithm still has a single loop that runs from 2 to n which is relative to n , so the total time complexity is $O(n)$.

□

5. (5 pts) In a table, list each of the four algorithms as rows and for each give its asymptotic time and space requirements. Briefly discuss how these different approaches compare, and where the improvements come from.

Answer. Comparing All Four Algorithms:

Algorithm	Time Complexity	Space Complexity
Recursive Top-Down	$O(2^n)$	$O(n)$
DP Top-Down	$O(n)$	$O(n)$
DP Bottom-Up	$O(n)$	$O(n)$
DP Bottom-Up $\Theta(1)$	$O(n)$	$\Theta(1)$

The DP Bottom Up with $\Theta(1)$ time complexity is the best algorithm because the time complexity is $O(n)$ and the space complexity is $\Theta(1)$. The decrease in space occurs from the fact that we only use a fixed number of variables instead of an array of size n . But like the DP Bottom-Up and DP Top-Down, the time complexity is $O(n)$ because there is a single loop that runs from $i = 2$ to n .

Followed by DP Bottom-Up and DP Top-Down which have the same time and space complexity of $O(n)$. The time complexity has decreased as opposed to the recursive top-down approach because we are storing the values of the recursions in a table so that we can avoid redundant computations.

□

6. (15 pts) Implement your algorithm in part (4) in Python and then compute F_n where n is the four-digit number representing your MMDD birthday, and report the first five digits of F_n . Now, assuming that it takes one nanosecond per operation, estimate the number of years required to compute F_n using the algorithm in part (1) and compare that to the time required to compute F_n using algorithm in part (4).

Answer.

```
def fibonacci(n):
    if n <= 1:
        return n
    x = 0
    y = 1
    for i in range(2, n):
        z = x + y
        x = y
        y = z

    return y
```

My birthday is February 22, so my $n = 0222$. The first five digits of $F_n = 68672$.

Using the algorithm in part 1, the time complexity is $O(2^n)$. So $2^{222} = 6.74 \cdot 10^{66}$ nanoseconds which is approximately **2.14** years.

Using the algorithm in part 4, the time complexity is $O(n)$ so $O(222)$ is 222 nanoseconds which is $7.04 \cdot 10^{-15}$. The time it takes to run the algorithm in part 4 is a lot faster than the algorithm in part 1.

□